1、使用 mkdir 创建目录文件夹，名称自己取

2、在自己创建的目录下，编写 vm.c

内核里面不能使用 printf()函数印信息，内核只能使用 printk()函数

/* printk 相当于 printf 的孪生兄弟，她们一个运行在用户态的，另一个则在内核态被人们所熟知的。但是根据不同的操作系统也会有不一样的效果，例如编写一个 hello word 内核模块，使用这个函数不一定会将内容显示到终端上，但是一定在内核缓冲区里，可以使用 dmesg 查看效果。

*/

```
#include<linux/init.h>

#include<linux/kernel.h>

#include<linux/module.h>


static int __init lvk_init(void)

{

    printk("Hello world Kernel.\n");

    return 0;
```

```
}

static void __exit lvk_exit(void)
{
    printk("Goodbye Kernel.\n");
}

module_init(lvk_init);
module_exit(lvk_exit);

MODULE_LICENSE("GPL"); /* 模块的许可证声明 */
```

3、打开另一个终端进程 idmod 目录，编写 Makefile 文件

Makefile 文件首字母必须大写，否则 系统找不到会报错。

```
obj-m:=vm.o

CURRENT_PAHT:=$(shell pwd)  /* 命令用于显示当前
```

工作目录的绝对路径 */

LINUX_KERNEL:=$(shell uname -r) /* uname -r : 显示操作系统的发行版号 */

LINUX_KERNEL_PATH:=/usr/src/linux-headers-$(LINUX_KERNEL)

all:

  make -C $(LINUX_KERNEL_PATH) M=$(CURRENT_PAHT) modules

clean:

  make -C $(LINUX_KERNEL_PATH) M=$(CURRENT_PAHT) clean

4、确保上述两个文件必须保存且无误

5、查看编写的程序头文件.h，在内核源码是否有匹配的文件

a.进入/usr/src 目录

b.uname -r 查看当前版本

```
Activities  Terminal  Tue 02:42
vico@ubuntu: /usr/src/linux-headers-5.4.0-73-generic/include/linux
File  Edit  View  Search  Terminal  Help
vico@ubuntu:~$ cd /usr/src
vico@ubuntu:/usr/src$ ls
linux-headers-5.4.0-72-generic  linux-hwe-5.4-headers-5.4.0-72
linux-headers-5.4.0-73-generic  linux-hwe-5.4-headers-5.4.0-73
linux-hwe-5.4-headers-5.4.0-42
vico@ubuntu:/usr/src$ uname -r
5.4.0-73-generic
vico@ubuntu:/usr/src$ cd linux-headers-5.4.0-73-generic
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic$ ls
arch     Documentation  init     kernel    Module.symvers  security  usr
block    drivers        ipc      lib       net             sound     virt
certs    fs             Kbuild   Makefile  samples         tools
crypto   include        Kconfig  mm        scripts         ubuntu
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic$ ls -l
total 1612
drwxr-xr-x 3 root root    4096 Apr 26 02:27 arch
lrwxrwxrwx 1 root root      39 Apr 16 07:44 block -> ../linux-hwe-5.4-headers-5.4.0-73/
block
lrwxrwxrwx 1 root root      39 Apr 16 07:44 certs -> ../linux-hwe-5.4-headers-5.4.0-73/
certs
lrwxrwxrwx 1 root root      40 Apr 16 07:44 crypto -> ../linux-hwe-5.4-headers-5.4.0-73
/crypto
lrwxrwxrwx 1 root root      47 Apr 16 07:44 Documentation -> ../linux-hwe-5.4-headers-5
```

c. 进入 include/linux 目录查看头文件是否存在：

init.h、kernel.h、module.h

```
Activities  Terminal  Tue 02:43
vico@ubuntu: /usr/src/linux-headers-5.4.0-73-generic/include/linux
File  Edit  View  Search  Terminal  Help
lrwxrwxrwx 1 root root      38 Apr 16 07:44 virt -> ../linux-hwe-5.4-headers-5.4.0-73/v
irt
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic$ cd include
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include$ ls
acpi          config  dt-bindings  kvm       media  pcmcia  scsi   target  vdso
asm-generic   crypto  generated    linux     misc   ras     soc    trace   video
clocksource   drm     keys         math-emu  net    rdma    sound  uapi    xen
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include$ cd linux
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include/linux$ ls -l
total 11772
-rw-r--r--  1 root root   1038 Nov 24  2019 8250_pci.h
-rw-r--r--  1 root root   2551 Nov 24  2019 acct.h
-rw-r--r--  1 root root   3154 Nov 24  2019 acpi_dma.h
-rw-r--r--  1 root root  37772 Apr 16 07:44 acpi.h
-rw-r--r--  1 root root   2142 Nov 24  2019 acpi_iort.h
-rw-r--r--  1 root root    674 Nov 24  2019 acpi_pmtmr.h
-rw-r--r--  1 root root   1832 Nov 24  2019 adb.h
```

d. 使用 ls -l 命令查看对应头文件是否存在

```
Activities  Terminal  Tue 02:45
vico@ubuntu: /usr/src/linux-headers-5.4.0-73-generic/include/linux
File  Edit  View  Search  Terminal  Help
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include/linux$ ls -l init.h
-rw-r--r-- 1 root root 9778 Nov 24  2019 init.h
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include/linux$ ls -l kernel.h
-rw-r--r-- 1 root root 35079 Nov 24  2019 kernel.h
vico@ubuntu:/usr/src/linux-headers-5.4.0-73-generic/include/linux$ ls -l module.h
-rw-r--r-- 1 root root 24922 Nov 24  2019 module.h
```

6、使用 ls 查看当前目录(会有两个文件即算成功)

```
vico@ubuntu:~/Desktop/rdmodule$ ls
Makefile  vm.c
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$
```

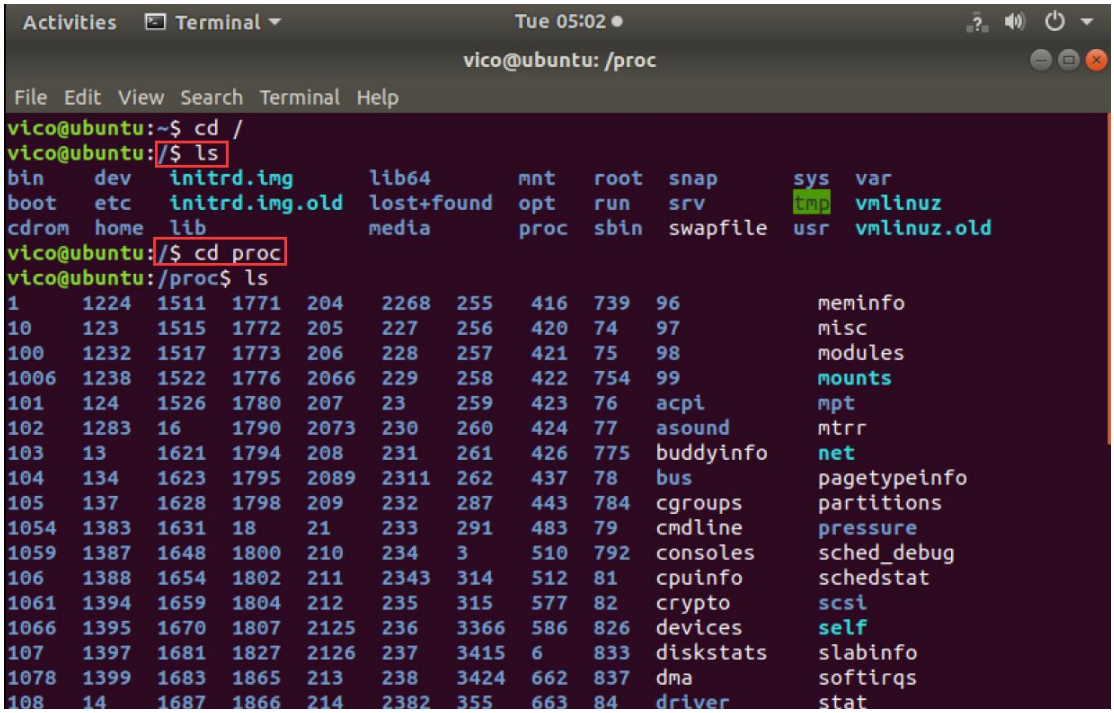
7、使用 make 命令（如下编译成功）

```
vico@ubuntu:~/Desktop/rdmodule$ make
make -C /usr/src/linux-headers-5.4.0-73-generic M=/home/vico/Desktop/rdmodule modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-73-generic'
  Building modules, stage 2.
  MODPOST 1 modules
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-73-generic'
vico@ubuntu:~/Desktop/rdmodule$ ls -l
total 36
-rw-rw-r-- 1 vico vico  254 May 25 04:53 Makefile
-rw-rw-r-- 1 vico vico   34 May 25 04:55 modules.order
-rw-rw-r-- 1 vico vico    0 May 25 04:54 Module.symvers
-rw-rw-r-- 1 vico vico  308 May 25 04:51 vm.c
-rw-rw-r-- 1 vico vico 4392 May 25 04:54 vm.ko
-rw-rw-r-- 1 vico vico   34 May 25 04:54 vm.mod
-rw-rw-r-- 1 vico vico  740 May 25 04:54 vm.mod.c
-rw-rw-r-- 1 vico vico 3128 May 25 04:54 vm.mod.o
-rw-rw-r-- 1 vico vico 2136 May 25 04:54 vm.o
vico@ubuntu:~/Desktop/rdmodule$
```

生成.ko 文件

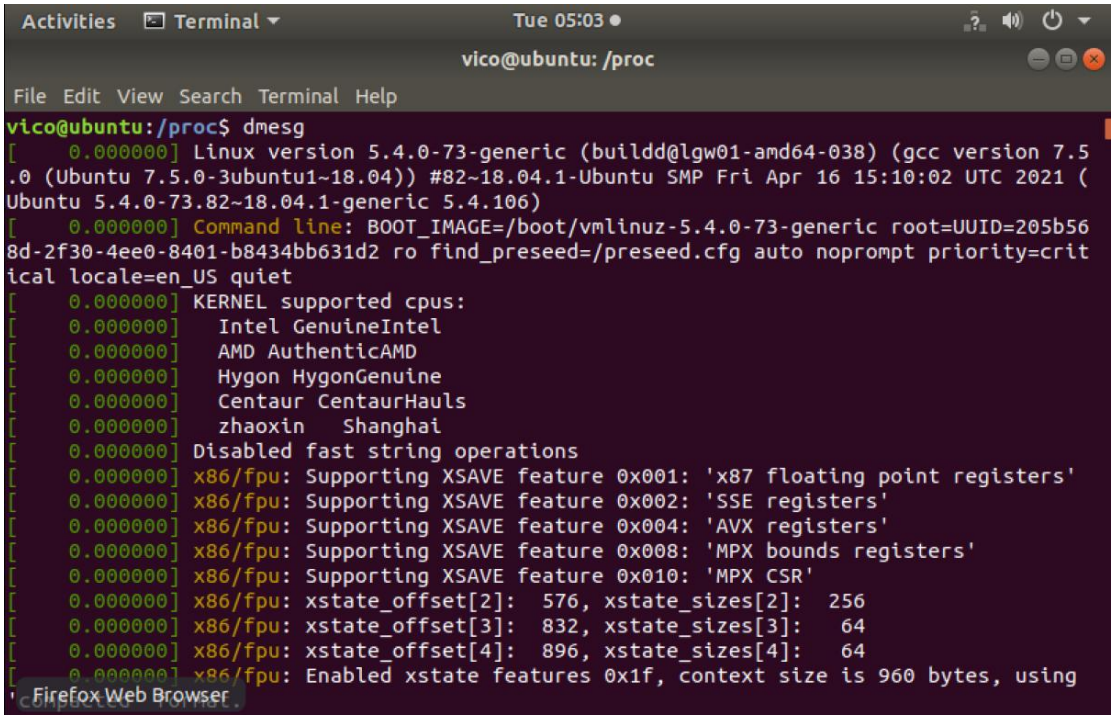
```
-rw-rw-r-- 1 vico vico 4392 May 25 04:54 vm.ko
```

8、进入 proc 目录（Linux 内核提供了一种通过 /proc 文件系统，在运行时访问内核内部数据结构、改变内核设置的机制）

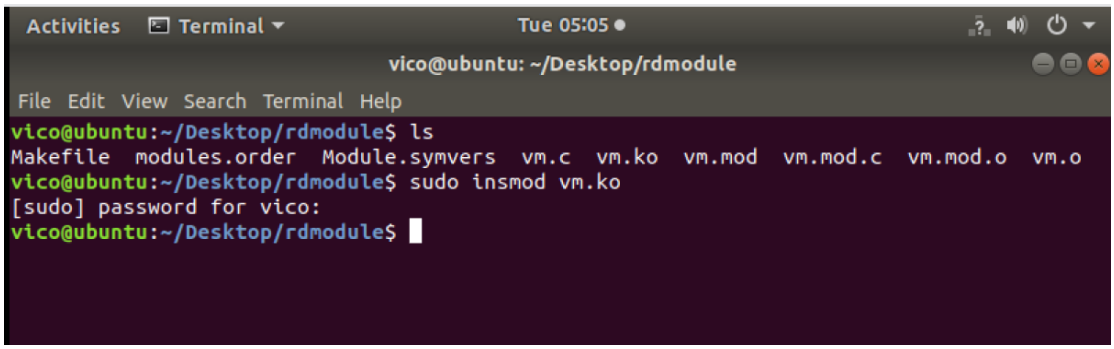


使用 dmesg 查看日志信息

dmesg 命令用于打印 Linux 系统开机启动信息，kernel 会将开机信息存储在 ring buffer 中。您若是开机时来不及查看信息，可利用 dmesg 来查看（print or control the kernel ring buffer）。开机信息亦保存在/var/log/dmesg 的文件里。某些硬件设备（比如七号信令卡、语音卡之类）在安装的时候，通常会安装驱动程序（内核模块），会打印一些信息，就可以通过 dmesg 命令来查看。



9、模块插入操作



10、lsmod 命令查看当前系统插入的模块信息

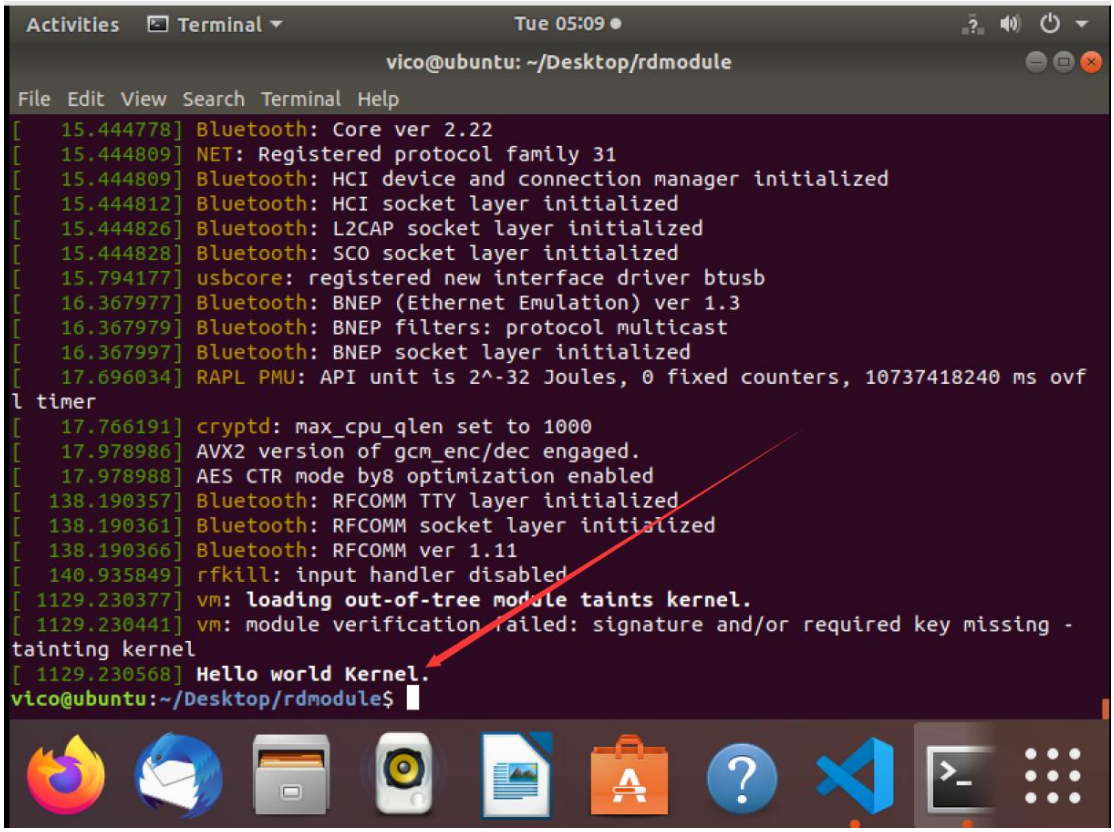
```
Activities  Terminal  Tue 05:07
vico@ubuntu: ~/Desktop/rdmodule
File  Edit  View  Search  Terminal  Help

vico@ubuntu:~/Desktop/rdmodule$ ls
Makefile  modules.order  Module.symvers  vm.c  vm.ko  vm.mod  vm.mod.c  vm.mod.o  vm.o
vico@ubuntu:~/Desktop/rdmodule$ sudo insmod vm.ko
[sudo] password for vico:
vico@ubuntu:~/Desktop/rdmodule$ lsmod
Module                  Size  Used by
vm                     16384  0
binfmt_misc            24576  1
rfcomm                 81920  4
intel_rapl_msr         20480  0
intel_rapl_common      24576  1 intel_rapl_msr
crct10dif_pclmul       16384  1
crc32_pclmul           16384  0
ghash_clmulni_intel    16384  0
vmw_balloon            24576  0
aesni_intel           372736  0
```

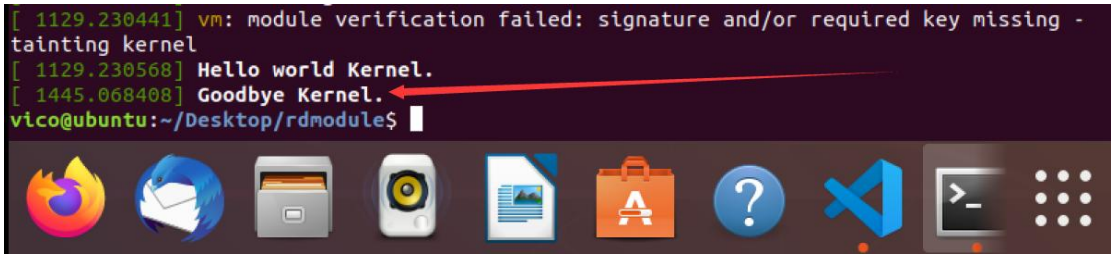
11、打印日志信息

```
Activities  Terminal  Tue 05:09
vico@ubuntu: ~/Desktop/rdmodule
File  Edit  View  Search  Terminal  Help
pata_acpi              16384  0
vico@ubuntu:~/Desktop/rdmodule$
vico@ubuntu:~/Desktop/rdmodule$ dmesg
[    0.000000] Linux version 5.4.0-73-generic (buildd@lgw01-amd64-038) (gcc version 7.5
.0 (Ubuntu 7.5.0-3ubuntu1~18.04)) #82~18.04.1-Ubuntu SMP Fri Apr 16 15:10:02 UTC 2021 (
Ubuntu 5.4.0-73.82~18.04.1-generic 5.4.106)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.4.0-73-generic root=UUID=205b56
8d-2f30-4ee0-8401-b8434bb631d2 ro find_preseed=/preseed.cfg auto noprompt priority=crit
ical locale=en_US quiet
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
```

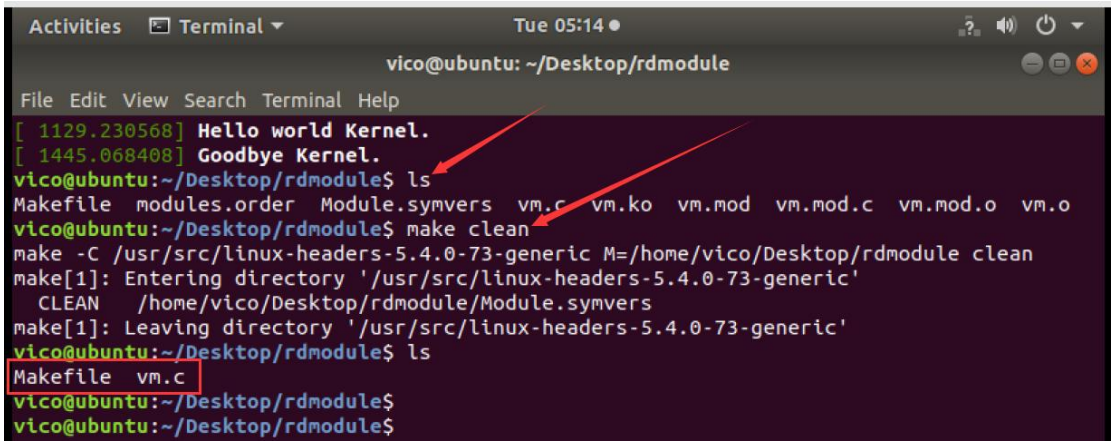
结果如下：



12、删除模块操作（sudo rmmod vm）不需要加.ko



13、使用 make clean 将编译文件进行清除操作



1. /proc 目录 Linux 内核提供了一种通过 /proc 文件系统，在运行时访问内核内部数据结构、改变内核设置的机制。proc 文件系统是一个伪文件系统，它只存在内存当中，而不占用外存空间。它以文件系统的方式为访问系统内核数据的操作提供接口。

用户和应用程序可以通过 proc 得到系统的信息，并可以改变内核的某些参数。由于系统的信息，如进程，是动态改变的，所以用户或应用程序读取 proc 文件时，proc 文件系统是动态从系统内核读出所需信息并提交的。下面列出的这些文件或子文件夹，并不是都是在你的系统中存在，这取决于你的内核配置和装载的模块。另外，在/proc 下还有三个很重要的目录：net，scsi 和 sys。Sys 目录是可写的，可以通过它来访问或修改内核的参数，而 net 和 scsi 则依赖于内核配置。例如，如果系统

不支持 scsi，则 scsi 目录不存在。

除了以上介绍的这些，还有的是一些以数字命名的目录，它们是进程目录。系统中当前运行的每一个进程都有对应的一个目录在/proc 下，以进程的 PID 号为目录名，它们是读取进程信息的接口。而 self 目录则是读取进程本身的信息接口，是一个 link。

2. 子文件或子文件夹/proc/buddyinfo 每个内存区中的每个 order 有多少块可用，和内存碎片问题有关

/proc/cmdline 启动时传递给 kernel 的参数信息

/proc/cpuinfo cpu 的信息

/proc/crypto 内核使用的所有已安装的加密密码及细节

/proc/devices 已经加载的设备并分类

/proc/dma 已注册使用的 ISA DMA 频道列表

/proc/execdomains Linux 内核当前支持的 execution domains

/proc/fb 帧缓冲设备列表，包括数量和控制它的驱动

/proc/filesystems 内核当前支持的文件系统类型

/proc/interrupts x86 架构中的每个 IRQ 中断数

/proc/iomem 每个物理设备当前在系统内存中的映射

/proc/ioports 一个设备的输入输出所使用的注册端口范围

/proc/kcore 代表系统的物理内存，存储为核心文件格式，里边显示的是字节数，等于 RAM 大小加上 4kb

/proc/kmsg 记录内核生成的信息，可以通过/sbin/klogd 或/bin/dmesg 来处理

/proc/loadavg 根据过去一段时间内 CPU 和 IO 的状态得出的负载状态，与 uptime 命令有关

/proc/locks 内核锁住的文件列表

/proc/mdstat 多硬盘，RAID 配置信息(md=multiple disks)

/proc/meminfo RAM 使用的相关信息

/proc/misc 其他的主要设备(设备号为 10)上注册的驱动

/proc/modules 所有加载到内核的模块列表

/proc/mounts 系统中使用的所有挂载

/proc/mtrr 系统使用的 Memory Type Range Registers (MTRRs)

/proc/partitions 分区中的块分配信息

/proc/pci 系统中的 PCI 设备列表

/proc/slabinfo 系统中所有活动的 slab 缓存信息

/proc/stat 所有的 CPU 活动信息

/proc/sysrq-trigger 使用 echo 命令来写这个文件的时候，远程 root 用户可以执行大多数的系统请求关键命令，就好像在本地终端执行一样。要写入这个文件，需要把 /proc/sys/kernel/sysrq 不能设置为 0。这个文件对 root 也是不可读的

/proc/uptime 系统已经运行了多久

/proc/swaps 交换空间的使用情况

/proc/version Linux 内核版本和 gcc 版本

/proc/bus 系统总线(Bus)信息，例如 pci/usb 等

/proc/driver 驱动信息

/proc/fs 文件系统信息

/proc/ide ide 设备信息

/proc/irq 中断请求设备信息

/proc/net 网卡设备信息

/proc/scsi scsi 设备信息

/proc/tty tty 设备信息

/proc/net/dev 显示网络适配器及统计信息

/proc/vmstat 虚拟内存统计信息

/proc/vmcore 内核 panic 时的内存映像

/proc/diskstats 取得磁盘信息

/proc/schedstat kernel 调度器的统计信息

/proc/zoneinfo 显示内存空间的统计信息，对分析虚拟内存行为很有用

以下是/proc 目录中进程 N 的信息

/proc/N pid 为 N 的进程信息

/proc/N/cmdline 进程启动命令

/proc/N/cwd 链接到进程当前工作目录

/proc/N/environ 进程环境变量列表

/proc/N/exe 链接到进程的执行命令文件

/proc/N/fd 包含进程相关的所有的文件描述符

/proc/N/maps 与进程相关的内存映射信息

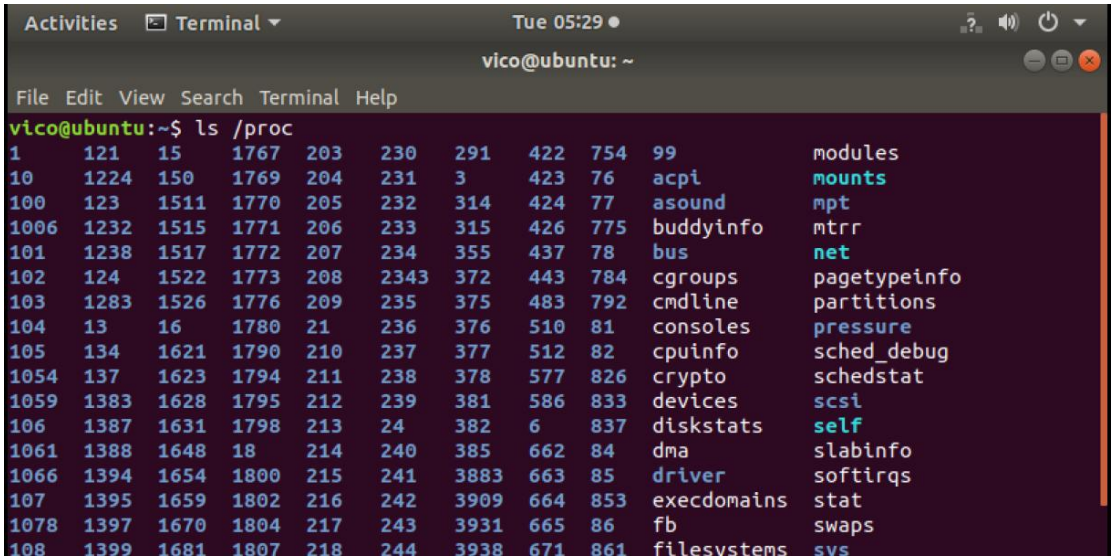
/proc/N/mem 指代进程持有的内存，不可读

/proc/N/root 链接到进程的根目录

/proc/N/stat 进程的状态

/proc/N/statm 进程使用的内存的状态

/proc/N/status 进程状态信息，比 stat/statm 更具可读性

/proc/self 链接到当前正在运行的进程

3. 例子 /proc/yafang@QA:~$ ls /proc/